CS344L - Artificial Intelligence Lab

LAB 05 TASK

Name: Hussnain Ahmad

Roll No: 242816

Class: BSCSeve-V-C

Course: Artificial Intelligence (CS344L)

Lab: Lab 5 - Constraint Satisfaction Problems

Task 1 - Exam timetable as a CSP (8 marks)

Six exams are scheduled into three days with backtracking search. Two exams sharing students (a clash) may not sit on the same day. The program counts slot tests and backtracks, then repeats the search with only two slots.

Code (242816_Task1.py):

```
"""
CS344L - Artificial Intelligence Lab | Lab 5 - Constraint Satisfaction Problems
Task 1 - Exam timetable as a CSP  (roll 242816)

CSP model (in my own words):
  Variables : the six courses that each need an exam slot.
  Domain    : the three days {Monday, Tuesday, Wednesday} - the same for
              every course.
  Constraint: two courses that share students (a clash) may not have their
              exams on the same day.
"""

COURSES = ["AI", "Database", "Networks", "OS", "Compiler", "Graphics"]

SLOTS = ["Monday", "Tuesday", "Wednesday"]

CLASHES = {
    "AI":       ["Database", "Networks", "OS"],
    "Database": ["AI", "Networks", "Graphics"],
    "Networks": ["AI", "Database", "OS", "Graphics"],
    "OS":       ["AI", "Networks", "Compiler"],
    "Compiler": ["OS", "Graphics"],
    "Graphics": ["Database", "Networks", "Compiler"],
}

tests = 0        # how many slot assignments were tried
backtracks = 0   # how many times a choice had to be undone


def reset_counters():
    # Set both counters back to zero before a new search.
    global tests, backtracks
    tests = 0
    backtracks = 0


def is_consistent(course, slot, assignment):
    # True only if no clashing course already holds that slot.
    for other, other_slot in assignment.items():
        if other in CLASHES[course] and other_slot == slot:
            return False
    return True


def backtracking_search(courses, slots):
    # Assign a slot to every course in list order. When no slot fits the
    # current course, undo the last choice (backtrack) and try its next slot.
    assignment = {}
```

```
    def backtrack(i):
        global tests, backtracks
        if i == len(courses):
            return True
        course = courses[i]
        for slot in slots:
            tests += 1
            if is_consistent(course, slot, assignment):
                assignment[course] = slot
                if backtrack(i + 1):
                    return True
                del assignment[course]  # undo the choice
                backtracks += 1
        return False

    if backtrack(0):
        return assignment
    return None


def print_timetable(plan):
    # Print one course per line, aligned in two columns.
    for course in COURSES:
        print(f"   {course:<10} {plan[course]}")


def main():
    # Solve once with three slots, then again with only two slots.
    print("Exam timetable (3 slots)")
    reset_counters()
    plan = backtracking_search(COURSES, SLOTS)
    print_timetable(plan)
    print(f"   tests: {tests}, backtracks: {backtracks}")
    print()

    print("With only 2 slots: ", end="")
    reset_counters()
    plan2 = backtracking_search(COURSES, SLOTS[:2])
    if plan2 is None:
        print("no valid timetable exists")
    else:
        print()
        print_timetable(plan2)
    print(f"   tests: {tests}, backtracks: {backtracks}")


if __name__ == "__main__":
    main()
```

Output: run window of Task 1

```
Task 1 - Exam timetable (backtracking)
$ python 242816_Task1.py
Exam timetable (3 slots)
   AI          Monday
   Database    Tuesday
   Networks    Wednesday
   OS          Tuesday
   Compiler    Wednesday
   Graphics    Monday
   tests: 15, backtracks: 1

With only 2 slots: no valid timetable exists
   tests: 10, backtracks: 4
```

Q1. Why is the two-slot version impossible? Name the three courses that make it impossible and explain in one line.

AI, Database and Networks clash pairwise with each other (every pair shares students), so those three courses need three different days - impossible with only Monday and Tuesday.

Q2. Your search undoes a choice at least once with three slots. Which course caused it, and why that one?

Graphics caused it: with the other five courses placed, all three days were already held by courses Graphics clashes with (Database on Tuesday, Networks on Wednesday, Compiler on Monday), so the search had to undo Compiler's slot. Counters: tests 15, backtracks 1.

Task 2 - MRV and degree heuristics (6 marks)

The Task 1 file is reused with the same is_consistent check; only the order changes. select_course() applies MRV (fewest legal slots left), breaking ties with the degree rule (most clashes with still-unscheduled courses).

Code (242816_Task2.py):

```
"""
CS344L - Artificial Intelligence Lab | Lab 5 - Constraint Satisfaction Problems
Task 2 - MRV and degree heuristics on the exam timetable  (roll 242816)

Same CSP as Task 1 (same variables, domain and clash constraint, same
is_consistent check). Only the order in which courses are picked changes:
plain backtracking takes the next course in the list, while the improved
search picks the most constrained course first (MRV + degree).
"""

COURSES = ["AI", "Database", "Networks", "OS", "Compiler", "Graphics"]

SLOTS = ["Monday", "Tuesday", "Wednesday"]

CLASHES = {
    "AI":       ["Database", "Networks", "OS"],
    "Database": ["AI", "Networks", "Graphics"],
    "Networks": ["AI", "Database", "OS", "Graphics"],
    "OS":       ["AI", "Networks", "Compiler"],
    "Compiler": ["OS", "Graphics"],
    "Graphics": ["Database", "Networks", "Compiler"],
}

tests = 0        # how many slot assignments were tried
backtracks = 0   # how many times a choice had to be undone


def reset_counters():
    # Set both counters back to zero before a new search.
    global tests, backtracks
    tests = 0
    backtracks = 0


def is_consistent(course, slot, assignment):
    # True only if no clashing course already holds that slot.
    # (Exactly the same check as in Task 1.)
    for other, other_slot in assignment.items():
        if other in CLASHES[course] and other_slot == slot:
            return False
    return True


def legal_slots(course, assignment):
    # The slots still possible for this course right now.
    return [s for s in SLOTS if is_consistent(course, s, assignment)]


def select_course(assignment):
```

```
    # MRV: pick the unscheduled course with the fewest legal slots left.
    # Tie-break with the degree rule: among those, pick the course that
    # clashes with the most courses that are still unscheduled.
    unscheduled = [c for c in COURSES if c not in assignment]
    best = None
    best_key = None
    for course in unscheduled:
        mrv = len(legal_slots(course, assignment))
        degree = sum(1 for other in CLASHES[course] if other in unscheduled)
        key = (mrv, -degree)  # fewer slots first, then higher degree
        if best_key is None or key < best_key:
            best = course
            best_key = key
    return best


def search(next_course):
    # Backtracking search where next_course(assignment) decides which
    # course to try next. Returns (assignment or None, pick order list).
    assignment = {}
    order = []  # (course, options left when it was picked)

    def backtrack():
        global tests, backtracks
        if len(assignment) == len(COURSES):
            return True
        course = next_course(assignment)
        options = legal_slots(course, assignment)
        order.append((course, options))
        for slot in SLOTS:
            tests += 1  # every slot tried counts, as in Task 1
            if is_consistent(course, slot, assignment):
                assignment[course] = slot
                if backtrack(
):
                    return True
                del assignment[course]  # undo the choice
                backtracks += 1
        order.pop()
        return False

    if backtrack():
        return assignment, order
    return None, order


def next_in_list(assignment):
    # Plain version: take the next course in the COURSES list.
    return COURSES[len(assignment)]


def main():
    # 1. Order chosen by the heuristics, with options left at each step.
    reset_counters()
    plan, order = search(select_course)
    print("Order chosen by MRV + degree:")
    for course, options in order:
        print(f"   pick {course:<10} options left: {options}")
    mrv_tests, mrv_backtracks = tests, backtracks
    print()
```

```
    # 2. Same problem with plain list order, for comparison.
    reset_counters()
    plan_plain, _ = search(next_in_list)
    plain_tests, plain_backtracks = tests, backtracks

    print("Comparison")
    print("   method                tests   backtracks")
    print(f"   plain backtracking    {plain_tests:<7} {plain_backtracks}")
    print(f"   MRV + degree          {mrv_tests:<7} {mrv_backtracks}")


if __name__ == "__main__":
    main()
```

Output: run window of Task 2

```
Task 2 - MRV + degree heuristics
$ python 242816_Task2.py
Order chosen by MRV + degree:
   pick Networks   options left: ['Monday', 'Tuesday', 'Wednesday']
   pick AI         options left: ['Tuesday', 'Wednesday']
   pick Database   options left: ['Wednesday']
   pick OS         options left: ['Wednesday']
   pick Graphics   options left: ['Tuesday']
   pick Compiler   options left: ['Monday']

Comparison
   method                tests   backtracks
   plain backtracking    15      1
   MRV + degree          12      0
```

Q3. Which course does MRV pick first, and why does the degree rule choose that one when all courses still have three slots available?

Networks. All six courses tie with three legal slots, so the degree rule breaks the tie - Networks clashes with 4 courses, more than any other, so it is picked first.

Q4. Your MRV version needs fewer tests than Task 1. Explain in two lines where the saving comes from.

MRV always places the most constrained course first (fail-first), so dead-end branches are found early instead of after testing many harmless combinations.

Here it avoided the one wasted detour entirely: 12 tests and 0 backtracks against 15 tests and 1 backtrack.

Task 3 - 8-Queens with your own roll number (6 marks)

Eight queens are placed by backtracking, one per column, with column 0 fixed to row (242816 mod 8) = 0. is_safe() checks the row and diagonal rules; a separate verify() checks every pair of the final board. An optional command-line row (python 242816_Task3.py 4) reruns with a forced first row for Q6.

Code (242816_Task3.py):

```
"""
CS344L - Artificial Intelligence Lab | Lab 5 - Constraint Satisfaction Problems
Task 3 - 8-Queens with your own roll number  (roll 242816)

CSP model (in my own words):
  Variables : the eight columns of the board (one queen per column).
  Domain    : the eight rows {0..7} - the same for every column.
  Constraints: (1) no two queens share a row;
               (2) no two queens share a diagonal - two queens share a
               diagonal exactly when their row difference equals their
               column difference.
               Plus the extra rule: the queen in column 0 is fixed to
               row (roll number mod 8).

Run:  python 242816_Task3.py            -> uses your own row (roll mod 8)
      python 242816_Task3.py 4          -> first queen forced into row 4
"""

import sys

N = 8
ROLL_NUMBER = 242816
START_ROW = ROLL_NUMBER % N  # 242816 % 8 = 0


def is_safe(column, row, assignment):
    # True only if a queen at (column, row) breaks neither the row rule
    # nor the diagonal rule against the queens already placed.
    for c, r in assignment.items():
        if r == row:
            return False
        if abs(r - row) == abs(c - column):
            return False
    return True


def solve(start_row):
    # Backtracking search with column 0 already fixed to start_row.
    # Tries rows 0..7 in each next column, undoing the last placement
    # (backtrack) when no row fits. Returns (assignment, placements, backtracks).
    assignment = {0: start_row}
    placements = 0   # rows tried
    backtracks = 0   # placements undone

    def backtrack(column):
        nonlocal placements, backtracks
        if column == N:
            return True
```

```
        for row in range(N):
            placements += 1
            if is_safe(column, row, assignment):
                assignment[column] = row
                if backtrack(column + 1):
                    return True
                del assignment[column]  # undo the placement
                backtracks += 1
        return False

    if backtrack(1):
        return assignment, placements, backtracks
    return None, placements, backtracks


def print_board(assignment):
    # Print the board using Q for queens and . for empty squares.
    print("     " + " ".join(str(c) for c in range(N)))
    for row in range(N):
        line = []
        for col in range(N):
            line.append("Q" if assignment.get(col) == row else ".")
        print(f"   {row} " + " ".join(line))
    print()


def verify(assignment):
    # Check every pair of queens; the board is safe only if no pair
    # shares a row or a diagonal.
    cols = sorted(assignment)
    for i in range(len(cols)):
        for j in range(i + 1, len(cols)):
            c1, c2 = cols[i], cols[j]
            r1, r2 = assignment[c1], assignment[c2]
            if r1 == r2 or abs(r1 - r2) == abs(c1 - c2):
                return False
    return True


def main():
    # Optional command-line row overrides the roll-number row (for Q6).
    star
t_row = int(sys.argv[1]) if len(sys.argv) > 1 else START_ROW

    print(f"Roll number: {ROLL_NUMBER}, "
          f"so column 0 queen is in row {start_row}")
    print()
    assignment, placements, backtracks = solve(start_row)
    print_board(assignment)
    print(f"   placements tried: {placements}")
    print(f"   backtracks      : {backtracks}")
    print(f"   verified safe   : {verify(assignment)}")


if __name__ == "__main__":
    main()
```

Output: run window of Task 3 (row 0)

```
Task 3 - 8-Queens (roll 242816, row 0)
$ python 242816_Task3.py
Roll number: 242816, so column 0 queen is in row 0

    0 1 2 3 4 5 6 7
  0 Q . . . . . . .
  1 . . . . . . Q .
  2 . . . . Q . . .
  3 . . . . . . . Q
  4 . Q . . . . . .
  5 . . . Q . . . .
  6 . . . . . Q . .
  7 . . Q . . . . .

  placements tried: 875
  backtracks      : 105
  verified safe   : True
```

Q5. How many boards would a brute-force method have to build for 8 queens if it placed one queen per column? Write it as a power and compare it with your placement count.

8^8 = 16,777,216 boards, against only 875 placements tried by backtracking - more than 19,000 times fewer, because inconsistent rows are never explored.

Q6. Run your program once more with the first queen in row 4 instead of yours. Report both placement counts and say in one line why a different starting row changes the amount of work.

Row 0 (roll 242816 mod 8): 875 placements, 105 backtracks. Row 4: 39 placements, 1 backtrack. A different fixed queen attacks different squares, so different amounts of the search tree are pruned before the search can waste work on them - row 4 happens to kill bad branches almost immediately.

Output: Task 3 rerun with first queen in row 4 (for Q6)

```
Task 3 - 8-Queens (first queen in row 4, for Q6)
$ python 242816_Task3.py 4
Roll number: 242816, so column 0 queen is in row 4

    0 1 2 3 4 5 6 7
  0 . Q . . . . . .
  1 . . . . . Q . .
  2 . . . . . . . Q
  3 . . Q . . . . .
  4 Q . . . . . . .
  5 . . . Q . . . .
  6 . . . . . . Q .
  7 . . . . Q . . .

  placements tried: 39
  backtracks      : 1
  verified safe   : True
```

Notes

All three files run on their own with plain Python 3 (standard library only, no packages installed). Full project on GitHub: https://github.com/hussnainahmedd/hussnain-ai-lab-5